



**Министерство науки и высшего образования Российской Федерации  
Мытищинский филиал федерального государственного автономного образовательного учреждения высшего образования  
«Московский государственный технический университет имени Н.Э. Баумана  
(национальный исследовательский университет)»  
(МФ МГТУ им. Н.Э. Баумана)**

**Факультет «Космический»  
Кафедра «Прикладная математика, информатика и вычислительная техника»  
Направление подготовки 09.03.01 - «Информатика и вычислительная техника»**

**Выпускная квалификационная работа на тему:**

**«Разработка нейросетевой модели для поддержки принятия решения при маршрутизации  
пожарной автотехники в городской среде»**

**Выполнил: Нистратов Радимир Алексеевич  
Руководитель: Вилисов Валерий Яковлевич**

# Задачи

01

## **Анализ систем маршрутизации**

Исследование факторов принятия решений диспетчером пожарной техники

02

## **Анализ RL алгоритмов**

Применение обучения с подкреплением в задачах экстренного реагирования

03

## **Математическая постановка**

Формализация задачи (MDP) и проектирование архитектуры нейросетевой СППР

04

## **Имитационное моделирование**

Разработка модели города в AnyLogic со случайными заторами и инцидентами

05

## **Программная интеграция**

Связь среды AnyLogic и Python через Pipeline для работы нейросетевого модуля

06

## **Интерфейсы UI/UX**

Создание панелей управления для диспетчера и администратора системы

## **07. Экспериментальное исследование и оценка эффективности**

Сравнительный анализ работы СППР с базовыми алгоритмами маршрутизации

# Основные задачи и требования к системам маршрутизации в пожарной службе

## Законодательное ограничение

Согласно ФЗ № 123-ФЗ, регламентированное время прибытия первого пожарного подразделения в городе не должно превышать 10 минут

## Основные оперативные задачи СППР:

**01**

Автоматическая  
верификация адреса и  
определение ранга  
сложности пожара

**02**

Идентификация доступных  
сил в ближайших гарнизонах

**03**

Расчет оптимальной  
траектории движения с  
учетом дорожных  
ограничений

# Критерии эффективности и ограничения

Главный критерий

**Минимизация интегрального времени реагирования**

$$T_{\text{реар}} \rightarrow \min$$

Вторичный критерий

**Минимизация пройденного расстояния**

$$S \rightarrow \min$$

# Анализ существующих методов поддержки принятия решений

---

## Классические детерминированные подходы

### Методы:

Алгоритмы Дейкстры, A\*, Флойда-Уоршелла, методы решения задач VRP

### Плюсы:

Математическая строгость и гарантированная сходимость точного оптимального пути на статическом графе

### Минусы:

Не способны адаптироваться к изменениям среды и распределять дефицитные ресурсы

## Интеллектуальные (нейросетевые) методы

### Методы:

Глубокое обучение с подкреплением (DRL), в частности алгоритмы DQN

### Плюсы:

Высокая скорость, учет пространственно-временных закономерностей города, предиктивное резервирование ресурсов

### Минусы:

Требуют длительного и ресурсоемкого процесса предварительной тренировки агента

# Архитектура системы

**Двухуровневая распределенная архитектура:** Создана для устранения конфликта между высокой детализацией симуляции и скоростью обучения нейросети.

## Исполнительный уровень

*AnyLogic / Java*

Микроскопическое агентное моделирование физических процессов, навигация, сбор телеметрии

## Интеграционный уровень

*шлюз Pipeline*

Бестаймаутный сокет-сервер для обмена данными по протоколу TCP/IP

## Ядро интеллекта

*Python / PyTorch*

Вычислительный модуль аппроксимации функций ценности действий

## Пространственная среда

ГИС-данные OpenStreetMap (OSM) в виде ориентированного графа с 4 депо. Внедрены зоны заторов (**jamZone**), агент обучается в условиях частичной наблюдаемости

# Теоретический базис: Марковский процесс принятия решений (MDP)

**Математическая формализация задачи:** Процесс последовательного выбора оптимальных диспетчерских решений в стохастической городской среде формализован как Марковский процесс принятия решений.

**Базовая структура процесса:**  $MDP = (S, A, P, R, \gamma)$  , где:

$S$  — конечное множество многомерных состояний среды;

$A$  — дискретное пространство допустимых управляющих воздействий;

$P$  — стохастическая матрица вероятностей переходов между состояниями;

$R$  — скалярная функция немедленного вознаграждения(штрафа);

$\gamma$  — коэффициент дисконтирования.

# Реализация марковского процесса принятия решений в СППР: Состояние, Действие и Награда

**Формализация состояния среды ( $S_t$ ):** Многомерная ресурсная и пространственная информация AnyLogic преобразуется в плоский нормализованный вектор:

- Категорию сложности текущего инцидента (**severity**)
- Вектор евклидовых расстояний от очага пожара до каждого из 4-х депо
- Матрицу остатка свободных ресурсов индивидуально по каждой станции (**station\_status**)

**Формализация пространства действий ( $A_t$ ):** Выбор конкретного номера (индекса) пожарной станции для отправки боевых расчетов на верифицированный инцидент

**Расчет функции вознаграждения ( $R$ ):** На основе фактического времени в пути:

$$R = -T_{\text{пути}} \cdot n,$$

где  $R$  — скалярный показатель отклика (значение вознаграждения или штрафа) виртуальной среды;

$T_{\text{пути}}$  — фактическое время пожарной техники в пути, полученное от модели симуляции;

$n$  — коэффициент скорости обучения ( $n = 10$ ).



# Структура передаваемых данных через шлюз Pipeline

## Вектор состояния (s)

```
{
  "fire_id": 1042,
  "severity": 1,
  "distances": [42, 127, 70, 321],
  "station_status": [
    [2, 1],
    [0, 0],
    [2, 1],
    [1, 0]
  ]
}
```

## Вектор действий (a)

```
{
  "incident_id": 1042,
  "selected_action": 1
}
```

# Протокол сопряжения и синхронизация времени

**Транспортное обеспечение:** Межпроцессное взаимодействие (IPC) на loopback (TCP/IP)

**Проблема времени:** Предотвращение искусственной задержки выезда техники за счет остановки виртуального времени AnyLogic во время работы нейросети

## Событийно-блокирующий алгоритм синхронизации

### 01. Пауза

Заморозка симуляции при новом инциденте:

```
getEngine().pause();
```

### 02. Экспорт

Формирование пакета состояния и отправка через шлюз Pipeline по TCP/IP

### 03. Расчет

Python обрабатывает JSON, выполняет forward-проход DQN и возвращает действие

### 04. Возобновление

Прием команды, отправка техники и разморозка:

```
getEngine().resume();
```

# Архитектура нейросети

## Глубокая Q-сеть (DQN)

### Полносвязный перцептрон (MLP):

- **Входной слой:** Размерность вектора состояния  $S_t$
- **Скрытые слои:** 2 слоя по 64 нейрона с активацией ReLU
- **Выходной слой:**  $n$  нейронов (Q-значения для станций)
- **Оптимизатор:** Adam (LR=0.001), функция потерь MSE

## Механизм обучения

- **Replay Buffer:** База данных из 5000 последних переходов для устранения автокорреляции
- **$\epsilon$ -greedy:** Баланс Exploration/Exploitation (decay 0.995)
- **Batch Training:** Обучение пакетами по 32 примера

# Реализация метода обучения

## Обновление весов

Вычисление критерия оптимизации и обновление весов нейросети автоматизированы в **PyTorch**

Используются стандартные высокоуровневые абстракции для эффективного градиентного спуска

```
def train_model():
    global current_loss, current_reward

    if len(memory) < BATCH_SIZE:
        return

    batch = random.sample(memory, BATCH_SIZE)
    states, actions, rewards = zip(*batch)

    states = torch.FloatTensor(np.array(states))
    actions = torch.LongTensor(actions).unsqueeze(1)
    rewards = torch.FloatTensor(rewards).unsqueeze(1)

    q_values = model(states).gather(1, actions)

    loss = nn.MSELoss()(q_values, rewards)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    current_loss = loss.item()
    current_reward = rewards.mean().item()

    print(f" [RL] Train Step! Loss: {current_loss:.4f} | Avg Reward: {current_reward:.2f} | Epsilon: {EPSILON:.2f}")
```

## Функция потерь (MSE)

Оценка расхождения выполняется с помощью встроенного класса **nn.MSELoss()**

Модуль автоматически рассчитывает отклонение между:

- Вектором текущих оценок полезности (выход Q-сети)
- Вектором временных целевых ориентиров (Target Q)

*Это обеспечивает стабильную сходимость алгоритма DQN*

# Гетерогенность ресурсов автопарка

fireTrucks - FireTruck

Имя:  ☒ Отображать имя ☐ Исключить

☐ Одиночный агент ☒ Популяция агентов

Популяция: ☐ Изначально пуста ☒ Содержит заданное кол-во агентов ☐ Загружается из базы данных

Начальное количество агентов:

type:

homeBase:

working: ☐

Настройка генерации гетерогенной популяции и визуализации машин в **AnyLogic**

80% ПАРКА

## Автоцистерна (TANKER)

Визуализация: **Красный цвет (Red)**

Базовый ресурс. Направляется на инциденты низкого ранга сложности

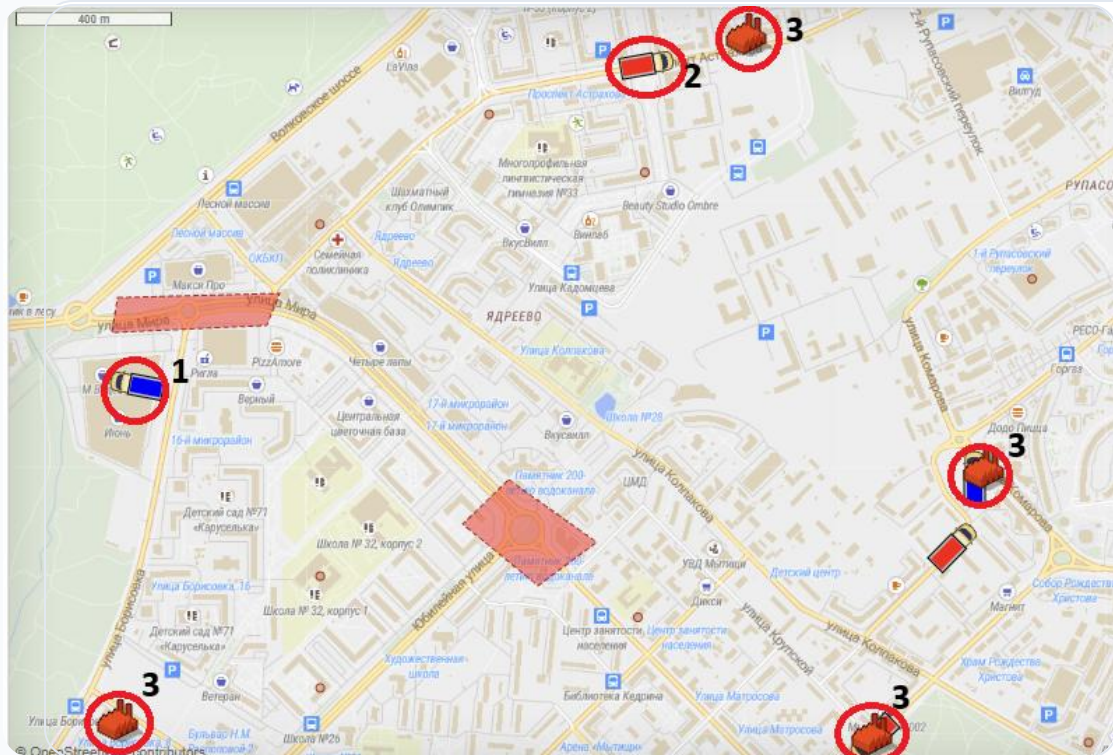
20% ПАРКА

## Автолестница (LADDER)

Визуализация: **Синий цвет (Blue)**

Дефицитный ресурс. Критически необходим для высотных инцидентов

# Объекты на карте имитационной модели



## Условные обозначения

1

**Автолестница**

Специализированная техника

2

**Автоцистерна**

Базовая пожарная техника

3

**Пожарные станции**

Места дислокации экипажей

# Роли пользователей и интерфейсы

## Диспетчер (ЛПР)

Лицо, принимающее решения в оперативной обстановке. Основной пользователь системы поддержки принятия решений (СППР)

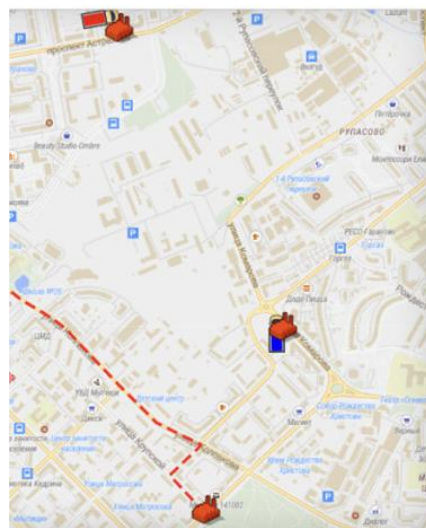
- Контроль инцидентов в реальном времени
- Подтверждение предложенных алгоритмом решений

## Администратор

Технический специалист, ответственный за конфигурацию системы и обучение моделей машинного обучения

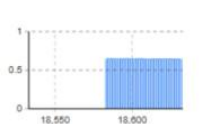
- Управление параметрами имитационной среды
- Дообучение и сохранение весов нейронной сети

# Интерфейс администратора



## Рабочее место администратора

- Частота возгораний
- Коэффициент загруженности дорог
- Скорость передвижения автотехники



Epsilon



Reward



Время реагирования



Функция потерь

к Диспетчер

Войти как Администратор

Сохранить веса

Загрузить веса

## Ключевые функциональные возможности:

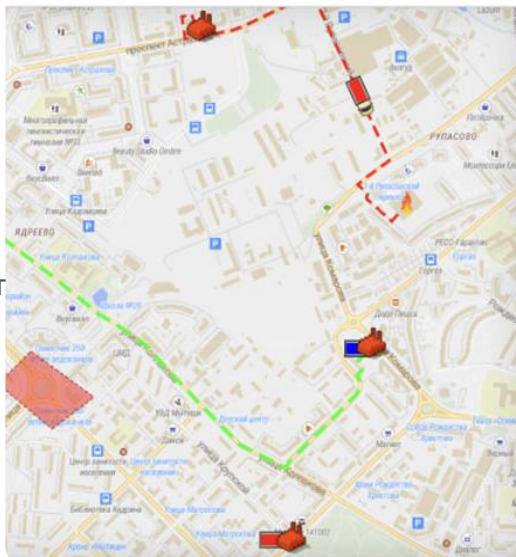
- Настройка параметров среды
- Мониторинг графиков обучения
- Сохранение/Загрузка весов модели



# Интерфейс диспетчера

## Ключевые функциональные возможности:

- Мониторинг времени реагирования и анимация маршрутов
- Управление СППР и рекомендации от DQN-модели
- Контроль резерва техники (АЦ/АЛ) по всем депо
- Выбор режима работы программы



### Рабочее место диспетчера

- ☐ Без ИИ  
☐ Автопилот ИИ  
☒ СППР (Интерактивный)

#### Статус пожарных станций:

Депо 1: АЦ - 1 шт. | АЛ - 1 шт.  
Депо 2: АЦ - 0 шт. | АЛ - 0 шт.  
Депо 3: АЦ - 1 шт. | АЛ - 0 шт.  
Депо 4: АЦ - 0 шт. | АЛ - 1 шт.

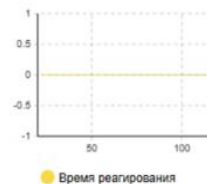
[ Отправить Депо 1 ]

[ Отправить Депо 2 ]

[ Отправить Депо 3 ]

[ Отправить Депо 4 ]

[ Подтвердить выбор ИИ ]



Рекомендация ИИ: Депо 1

# Методология и условия проведения экспериментов

## Цель исследований:

Сравнительная оценка эффективности разработанной СППР на базе алгоритма DQN и классических диспетчерских эвристик в ГИС-среде города

## Обеспечение чистоты эксперимента:

### В среде AnyLogic:

Фиксация генератора случайных чисел гарантирует абсолютно идентичный поток, координаты и сложность пожаров во всех тестах

### В среде Python:

Комплексная функция `seed_everything(seed = 1)` для полной детерминированности весов сети, буфера памяти и выбора действий

## Объем выборки и равенство условий:

- Собрано по **6000** последовательных измерений для каждого запуска
- Оба алгоритма получают на вход одинаковую изначальную матрицу евклидовых расстояний
- Состав автопарка зафиксирован: **80%** автоцистерн (TANKER) и **20%** автолестниц (LADDER)

# Сценарии моделирования

## Инструмент управления:

Параметры внешней среды регулируются с помощью трех интерактивных слайдеров:

- Вероятность пожара ( $p$ ): от 0 до 1
- Загруженность сети ( $k$ ): от 1 до 10
- Скорость техники ( $v$ ): от 1 до 100 у.е.

## Сценарий № 1

### (Штатный режим)

Умеренная интенсивность инцидентов и относительно свободная дорожная сеть

#### Параметры:

$p = 0.2$ ,  $k = 4$ ,  $v = 60$

## Сценарий № 2

### (Экстремальные условия)

Имитация часов пик, неблагоприятной погоды и резкого роста числа ЧС

#### Параметры:

$p = 0.6$ ,  $k = 8$ ,  $v = 40$

# Описание сравниваемых алгоритмов

## 1. Базовый алгоритм (ближайший по радиусу)

Реализует классический «жадный» пространственный поиск с использованием ГИС-функции для расчета расстояния по прямой

Направляет экипаж из ближайшего депо с любой свободной машиной

**Критический недостаток:** Алгоритм «слеп» к типам машин (`hasFreeTruck = true`). Это ведет к растрате дефицитных автолестниц на простые вызовы

## 2. Нейросетевой алгоритм (DQN-агент)

Принимает решения на базе вектора состояний с матрицей расстояний и учетом типов машин (`stationStatus`)

Соотносит евклидовы расстояния с реальным временем движения, адаптируясь к зонам заторов (`jamZones`)

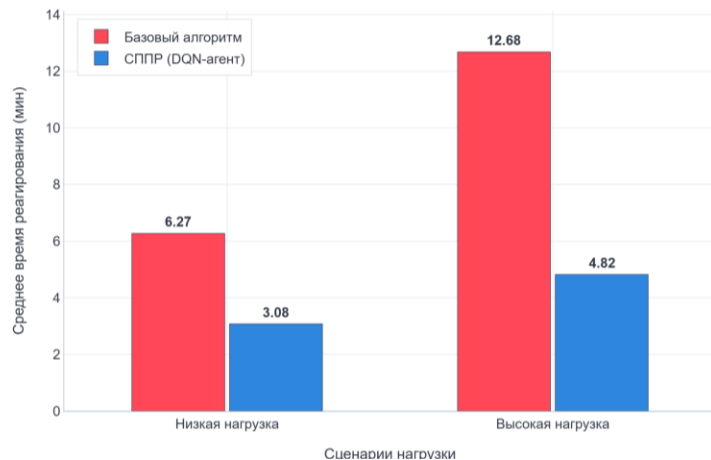
Учится стратегически «придерживать» автолестницы для сложных высотных вызовов (Ранг 2)

# Анализ результатов вычислительных экспериментов

Среднее время реагирования у СППР DQN меньше на ~50% при низкой нагрузке. При высокой - на ~62%

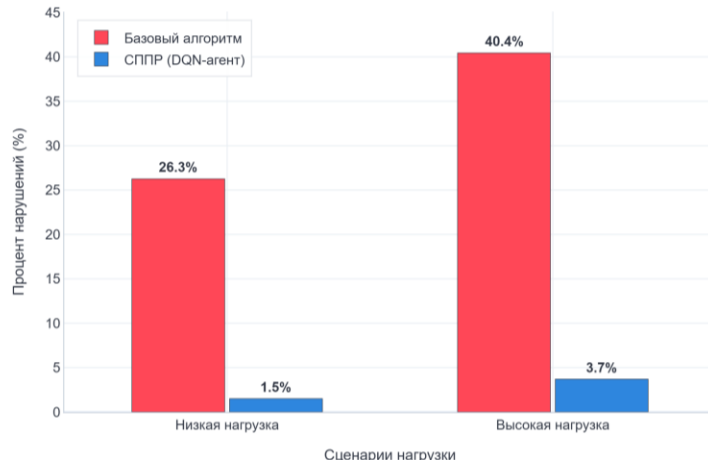
## Сравнение среднего времени реагирования

Сравнение среднего времени реагирования



## Доля выездов с нарушением регламента (> 10 мин)

Доля выездов с нарушением регламента (> 10 мин)



# Результаты моделирования: Сценарий № 1 (Низкая нагрузка)

Плотность распределения СППР DQN имеет ярко выраженную правостороннюю асимметрию с острым пиком в безопасном диапазоне 2.0 - 3.5 минут

Распределение «жадного» алгоритма сильно размыто по оси времени и имеет длинное правое плечо, уходящее в зону задержек

## Ключевые показатели

90-й перцентиль прибыли:

**Глубокая Q-сеть (DQN):  
4.0 мин**

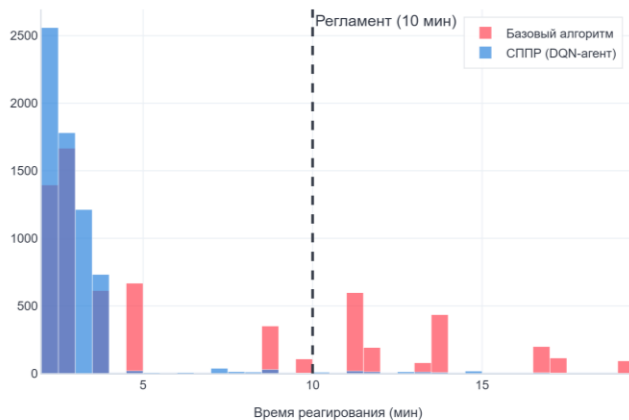
Базовый: 13.6 мин

Соблюдение регламента:

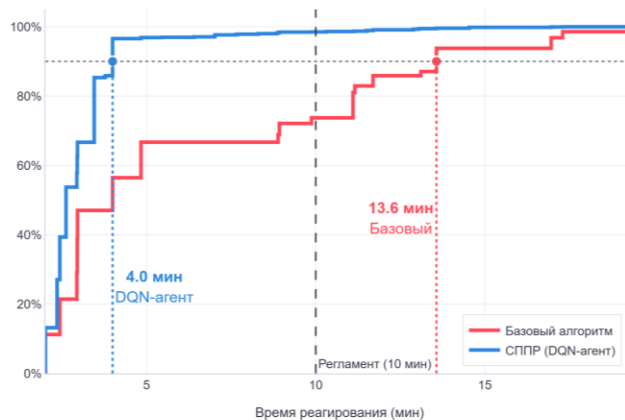
Интегральная  
вероятность (<10 мин)  
для глубокой Q-сети  
(DQN): **98.49%**

Доля нарушений: всего  
1.51%

Распределение времени реагирования (Низкая нагрузка)



Кумулятивное распределение времени реагирования (Низкая нагрузка)



# Результаты моделирования: Сценарий № 2 (Высокая нагрузка)

Плотность распределения «жадного» алгоритма полностью уплощается и приобретает тяжелый хвост вплоть до максимума в **38.14** мин. Время прибытия становится непредсказуемой величиной. СППР DQN эффективно противодействует стохастическим заторам, сохраняя плотный пик распределения в пределах **5 минут**

## Ключевые показатели

90-й перцентиль прибытия:

**Глубокая Q-сеть (DQN): 5.2 мин**

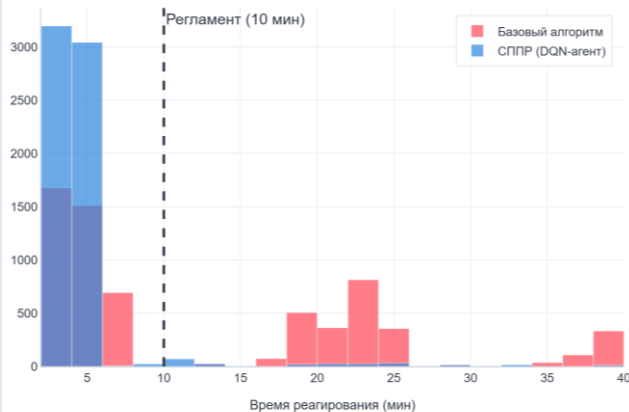
Базовый: 25 мин

Соблюдение регламента:

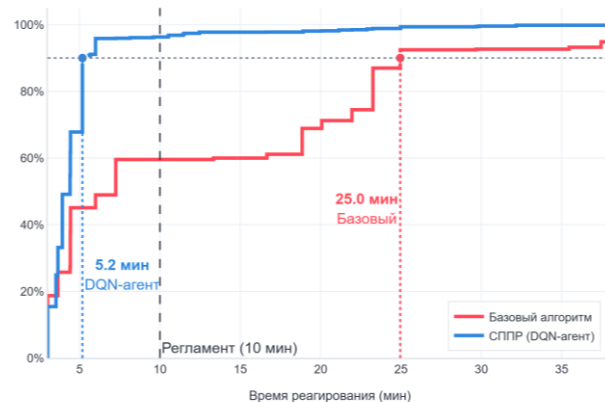
Вероятность (<10 мин) для глубокой Q-сети (DQN): **96.7%**

Доля нарушений: всего 3.7%

Распределение времени реагирования (Высокая нагрузка)



Кумулятивное распределение времени реагирования (Высокая нагрузка)



# Статистическая стабильность и анализ размаха данных

## Межквартильный размах (IQR)

**СППР (DQN-агент):** жестко сжат в узких пределах:

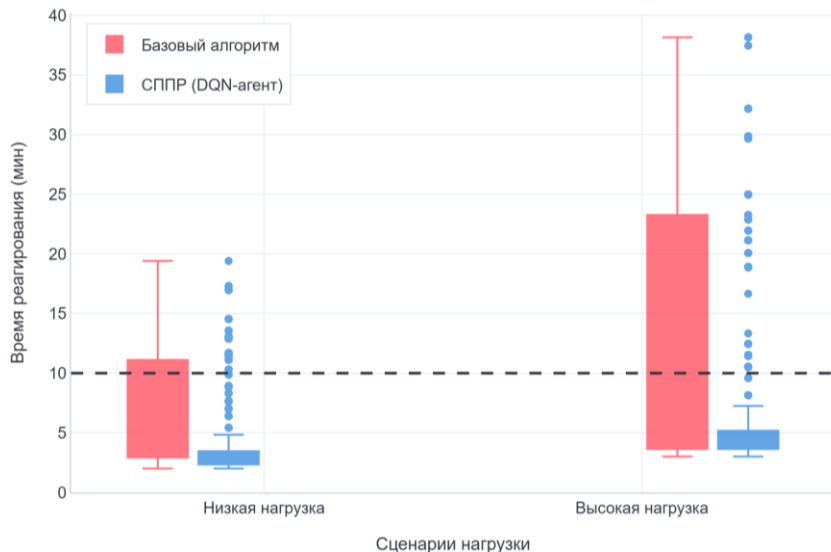
- Низкая нагрузка: [2.2, 3.4] мин
- Высокая нагрузка: [3.2, 5.0] мин

**Это доказывает высокую однородность и стабильность решений нейросети**

### Базовый алгоритм:

Тело ящика значительно шире, а верхний ус при высокой нагрузке уходит далеко за предел в 20 минут

Сравнение размаха и стабильности времени реагирования





# Результаты работы

## **Спроектированы два интерфейса**

Реализованы рабочее место диспетчера и администратора

## **Обучена модель DQN**

На базе MDP реализована стратегия предиктивного резервирования дефицитных ресурсов

## **Разработан комплекс СППР**

Создана распределенная архитектура, объединяющая среду и нейросетевое ядро через шлюз Pipeline

# Результаты работы (продолжение)

## Эффективность системы

### Время реагирования

Среднее время реагирования СППР лучше базового алгоритма:

В штатном режиме на ~50%, при высокой нагрузке на ~62%

### Доля нарушений регламента (<10 мин)

Базовый алгоритм имеет 26% при низкой нагрузке и ~40% при высокой

СППР же 1,51% и 3,70% соответственно

# Спасибо за внимание

---